AI4R System Project: team logbook

Ciaran Gruber, Thanh Long Nguyen, Zhenling Zhang, Peiyu Wang, Toby Lee, Yuhao Liu, Chris (Chuhan) Wang, Yifei Xia

Contents

- Part 0: Team overview
  - 0.1 Goals and requirements
  - 0.2 What ran on real cars
  - 0.3 The system and who built it
  - 0.4 Facts every part uses
- Part 1: Interfaces
  - 1.1 Ciaran Gruber — data model, logging and replay
- Part 2: Perception
  - 2.1 Shared: from sensors to the lane and the obstacles
  - 2.2 Thanh Long Nguyen — lane estimation, camera pan and the car simulator
  - 2.3 Zhenling Zhang — cone filter, obstacles, avoidance and local map
- Part 3: Movement policy (MPC)
  - 3.1 Shared: the MPC all three members build on
  - 3.2 Peiyu Wang — MPC base, MPC vs RL, virtual debug, keeping the code and cars working
  - 3.3 Toby Lee — test harness, MPC tuning, lane side, marker actions and intersection turns
  - 3.4 Yuhao Liu — measured-speed MPC, cone memory and junction planning
- Part 4: Car control
  - 4.1 Shared: the car model, controller and limits everyone uses
  - 4.2 Chris (Chuhan) Wang — system ID, speed fix and avoidance validation
  - 4.3 Yifei Xia — control tests, marker recognition and camera pan

Part 0: Team overview

Parts follow the sub-teams in pipeline order: 1 interfaces, 2 perception, 3 movement policy (MPC), 4 car control. Each part opens with what its members share, then one section per person: **S1** where it fits, **S2** how it works, **S3** simulation, **S4** real car. Results are labelled **car**, **recording** or **sim**. The full per-person sections are in `docs/logbook/<name>.md`.

0.1 Goals and requirements

The policy runs once per cone batch (≈ 10 Hz) on a 1/10 car on DREAM: cones, lidar, wheel speed, IMU and ArUco markers in; speed, curvature and camera-pan set-points out. It never enables the vehicle.

- **Stage 1:** follow a straight cone lane, hold 0.5 m/s, stop at the lane end and when the lane is lost. MPC and RL were compared before choosing the MPC (milestone M1, 3.2).
- **Stage 2:** curves and other widths, obstacles, markers that set the lane side, turns at intersections.

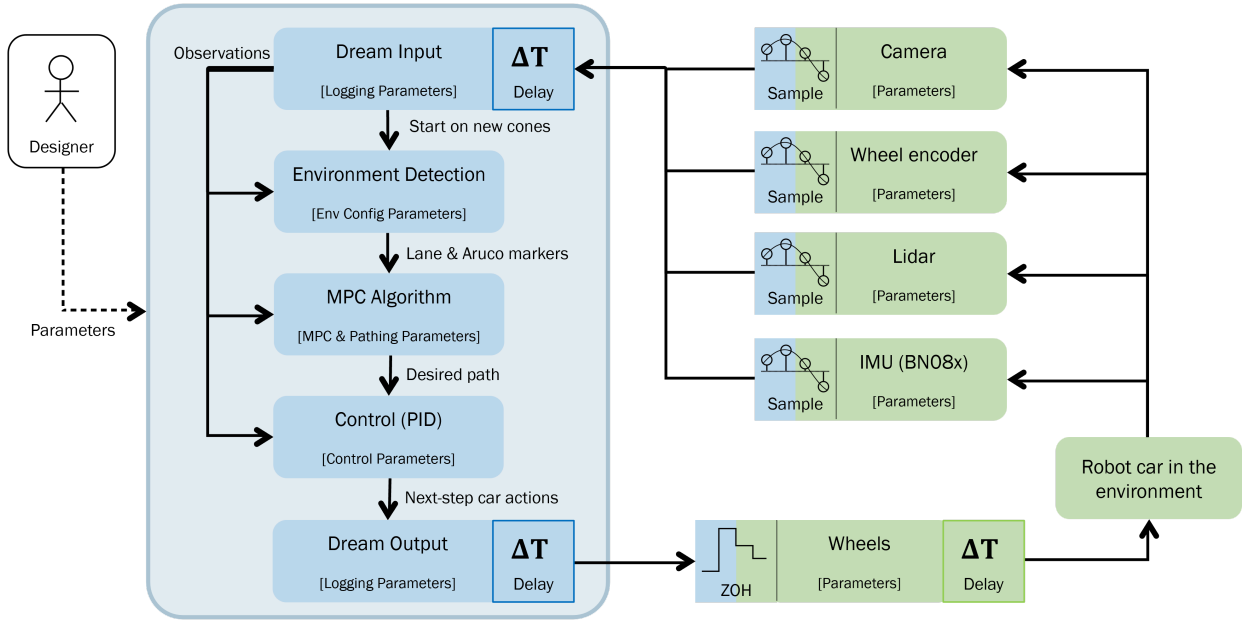
Id	Requirement (`tests/policy/stage1_scenarios.py`)	Pass if
R1a / R1b	Body inside the cone rows / distance from centre at the last pair	0.0 m / ≤ 0.10 m
R2	Cones touched	0
R3	Mean speed error (1 m after start to 1 m before end)	≤ 0.10 m/s
R4	RMS rate of change of requested curvature	≤ 1.0 1/(m s)
R5	Stop position past the last cone pair	0.0–0.5 m
R6	p99 step time on the car / solver failures	≤ 20 ms / 0
R7	Lane lost → stop request	≤ 0.6 s

0.2 What ran on real cars

Behaviour	Car, date	Result	See
Speed control at 0.5 m/s	20, 5 Oct	Feedforward + PI: 15% overshoot, ≈ 0.013 m/s steady error	4.2
MPC inside the time budget	20, 6 Oct	0.46 ms median solve against 20 ms	3.2
Lost-lane stop	22, 12 Oct	Stopped 0.8 s after losing the lane	3.2
Keep to the left of the lane	20, 20 Oct	Kept left (by eye, no log)	3.3
Obstacle stops	22 and 26 Oct	5 stops, 0.21–0.33 m short of the box	2.3
Passing obstacles	20, 26 Oct	Passed a box right, a box left, stopped 0.23 m short of a blocking box	2.3
Speed fault fix	20, 26 Oct	3 runs settled at 0.50–0.54 m/s (before: 0.87 m/s or no start)	4.2
Marker IDs from the live detector	19 Oct	`ids=[2]` once the camera image reached the detector	4.3
Marker command	13, 27 Oct	"Marker 14: KEEP_RIGHT", 6 times	3.3
Left turn at an intersection	20, 30 Oct	Marker read after a detector reload; turn refused as geometrically infeasible	3.4

Simulation only: a complete intersection turn. **Videos (real cars):** best video, 26 Oct obstacle drive (2.3); also the 15 Oct two-board stop (4.2). **AI use:** members used AI tools for code, figures and drafting, as the FEIT policy allows for the logbook; every result comes from the named commits, logs and runs.

0.3 The system and who built it



The policy (grey) runs once per cone batch; each block's parameters come from `config/`. Sensors and wheels each add a delay; the wheels hold the last command (ZOH).

Block (sub-team)	Person	Built
Dream input/output (interfaces)	Ciaran Gruber	Data model, logging, replay, `CarMemory`, repo, CI
Environment detection (perception)	Thanh Long Nguyen	Lane estimation, pan hold, whole-system simulator
	Zhenling Zhang	Cone filter, lidar obstacles, avoidance planner, obstacle stop, local map
MPC and pathing (movement policy)	Peiyu Wang	MPC framework, Stage 1 design (MPC vs RL), lane end, car debugging, Foxglove
	Toby Lee	Test harness, MPC settings for car 20, lane side, marker actions, intersection turner, left-turn fix (branch)
	Yuhao Liu	Measured-speed MPC, cone-memory U-turn, junction path finding, marker IDs and turn preparation, stop latch, car 20 diagnosis
Control (car control)	Chris (Chuhan) Wang	System ID, `CarController`, speed fix, avoidance validation
	Yifei Xia	Control tests with Chris, ArUco ID tests, camera pan calibration and simulation

0.4 Facts every part uses

Topic	Fact
Frames	`base_link`: +x forwards, +y left; lateral error, heading error and curvature positive to the left
Cone detections	≈ 10 per second, ≈ 0.2 s old (0.197 s measured on car 20)
Car 20 steering reach	+0.48 1/m left, −0.92 1/m right at ±0.5 steering (5 Oct). Left limit 0.91 1/m at ±0.8 is not ground-measured
Marker IDs (`dev`)	1–4 turn left, 5–6 turn right, 7–8 straight, other IDs stop; keep-side IDs used 23–27 Oct
Camera pan	Held at 0 since 10 Oct (2.2)
Stops	Obstacle stop brakes (drive −0.3, ≤ 1 s); lane-end stop plans a coast; roll-out not measured

Part 1: Interfaces

1.1 Ciaran Gruber — data model, logging and replay

Built: the observation/action data model between DREAM and the policy, data logging, scenarios from logs and their renderer, `CarMemory` (state carried between steps: lane side, marker command, stops, local map), the repo, CI and the car upload script. **PRs** #3 #4 #5 #7 #13 #20 #22.

Standard 1: Explanation to be done in interactive discussion with block diagram in 0.3 The system and who built it.

Standard 2.: Most evidence for this is observed through the design of the overall policy architecture.

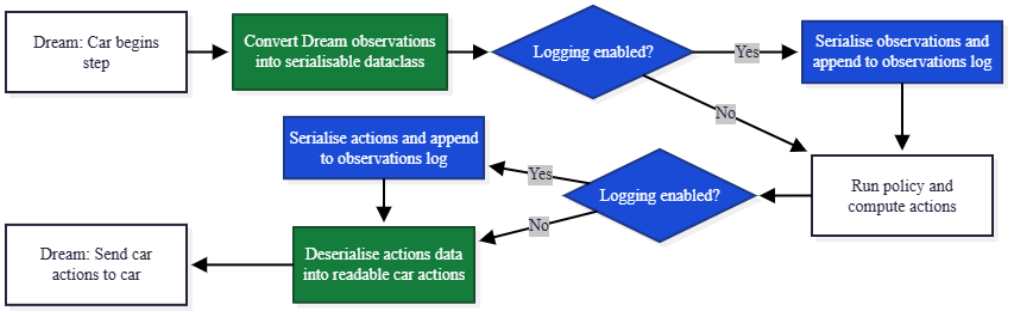


Figure: The algorithm used for managing logging in the car

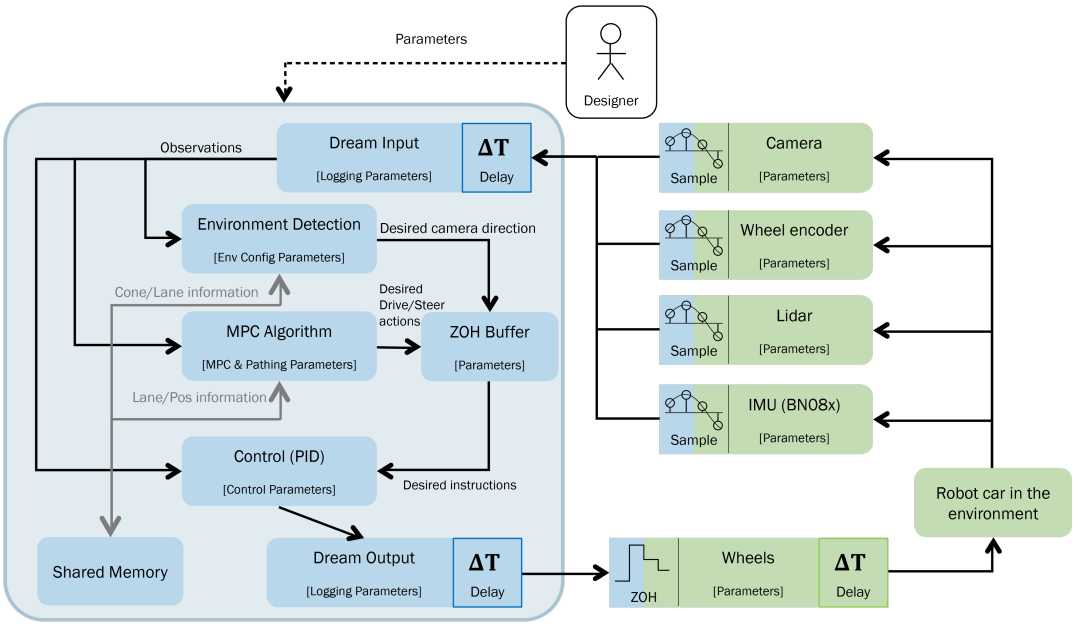


Figure: Modification of program architecture designed for concurrent execution of policy nodes.

Standard 3. Using single-step simulation to ensure consistent behaviour during architectural redesigns

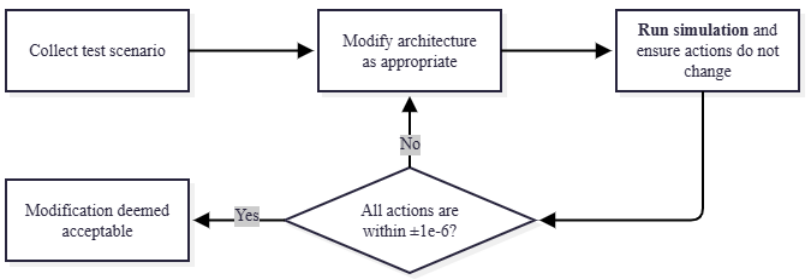


Figure: The process followed when making significant architectural changes involving single-step simulation to ensure every action matches within 10^{-6}.

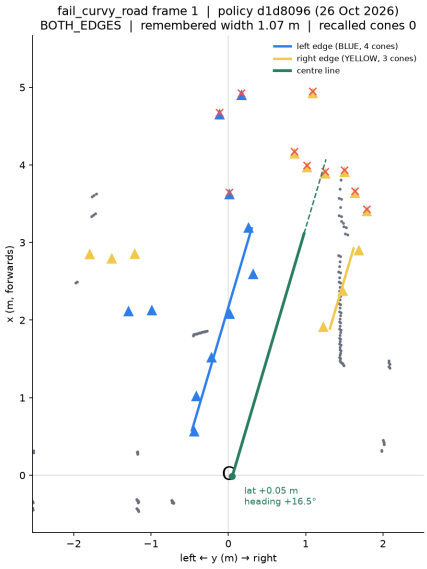
Standard 4: Resolving issues with car movement

Engineering question: Why does the car veer into the cones on curved tracks? (See figure below)

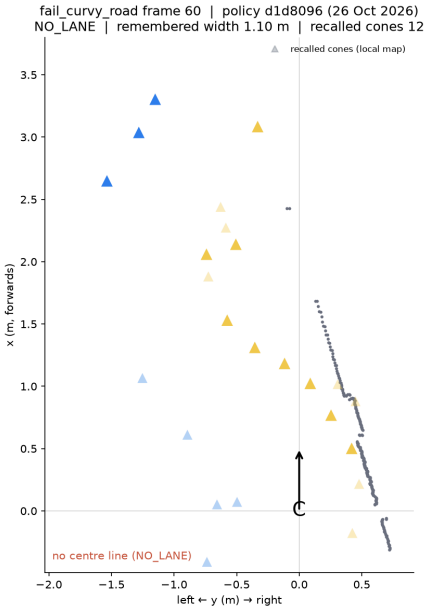
Unit tests and simulation passed, but on the curvy track the car hit cones. Logged the run, converted it to a scenario (`fail_curvy_road`), replayed it with that day's policy (`d1d8096`) and config, and overlaid the lane the policy produced. Obstacle seen in the below figure was removed to isolate curvature.



Track (obstacle later removed)



Frame 1



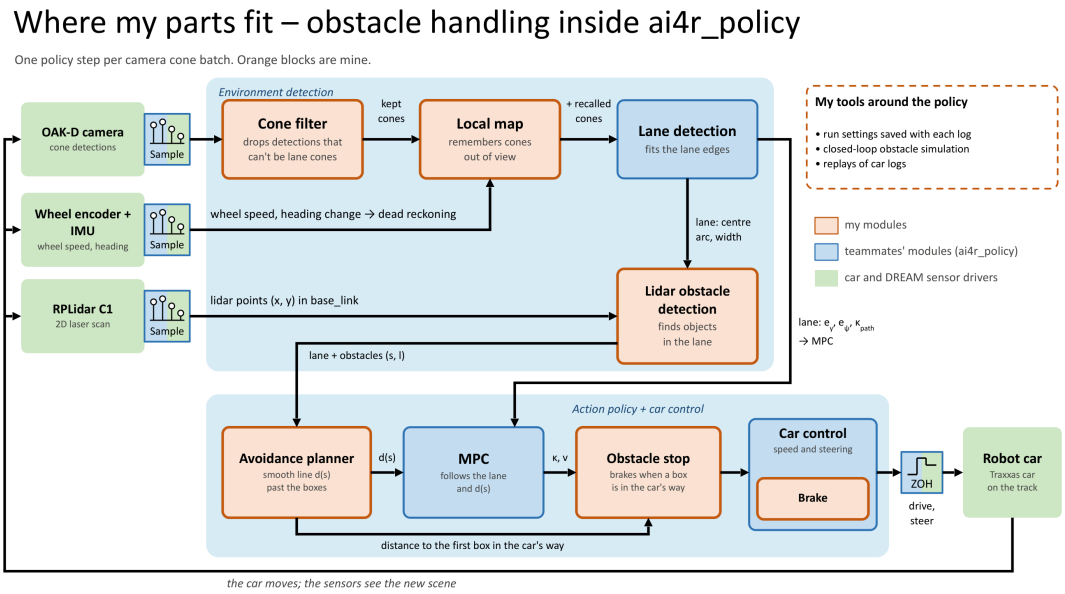
Frame 60: `NO_LANE`

Far blue cones conflict with the near yellow row at frame 60 and the detector reports `NO_LANE`.

→ Lane keeping now follows the yellow row when the blue row is missing (2.2); gentler curvature also helped.

Part 2: Perception

2.1 Shared: from sensors to the lane and the obstacles



One step per cone batch. Cones → filter (2.3) → lane estimate (2.2) → MPC (3.1). Lidar points become obstacles only once the lane is fitted (2.3); every output reaches the car through the MPC and car control.

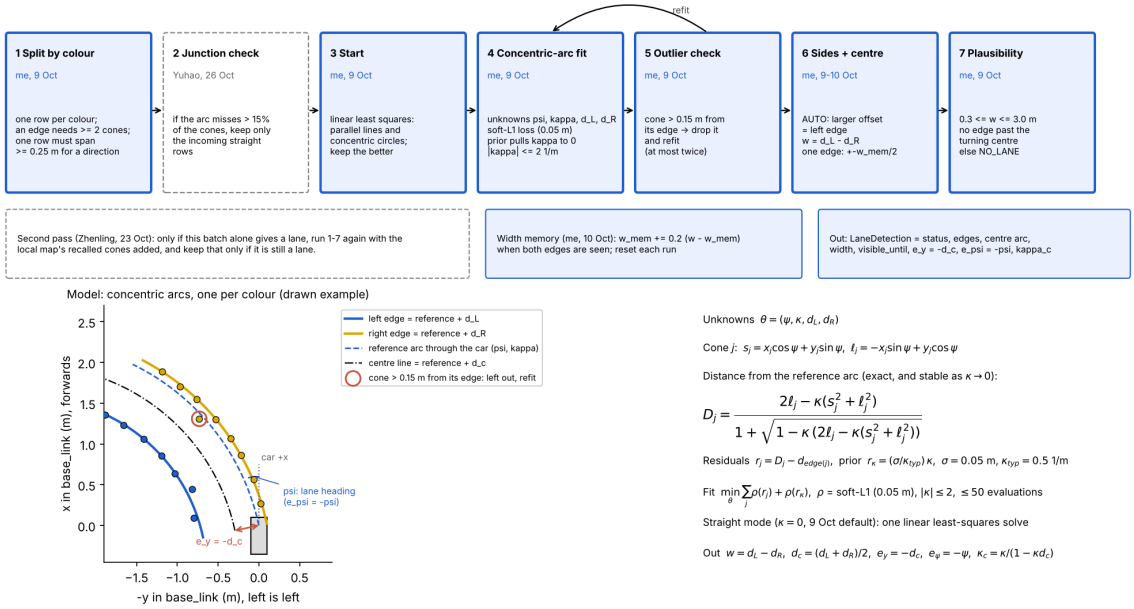
Interface	Carries	Constraint
Cones in	(x, y) in `base_link`, colour, confidence	≈ 10 Hz, 0.2 s old; duplicates, floating detections
Lidar in	points in `base_link`	New scan every 0.2–0.3 s; plane ≈ 0.12 m up, so it hits cones
`LaneDetection` → MPC, obstacles, turner	e_y (m), e_ψ (rad), κ_c (1/m), width, centre arc	`NO_LANE` stops the car; camera must point straight ahead
`LanePath` → MPC	offset $d(s)$ from the centre to follow	Continuous curvature
Obstacle distance → stop	m from the car's front	Front taken 0.30 m ahead of `base_link`

Shared consequence: the lane estimate is the frame everything else uses. A 0.35 m jump in it moved an obstacle into the path and stopped car 20 (4.2); a camera yaw δ shifts the car ≈ 1.2 cm per degree and breaks turns (2.2).

2.2 Thanh Long Nguyen — lane estimation, camera pan and the car simulator

Built: `LaneDetector` and its `LaneDetection` output, AUTO colour sides and width memory, the pan hold, and a whole-system simulator. Commits `8b71f8b` `f96ee66` `f49333d` `3319051` (#19). Car 20 drove on it 9 Oct; in every run since 14 Oct.

S2: model and fit.



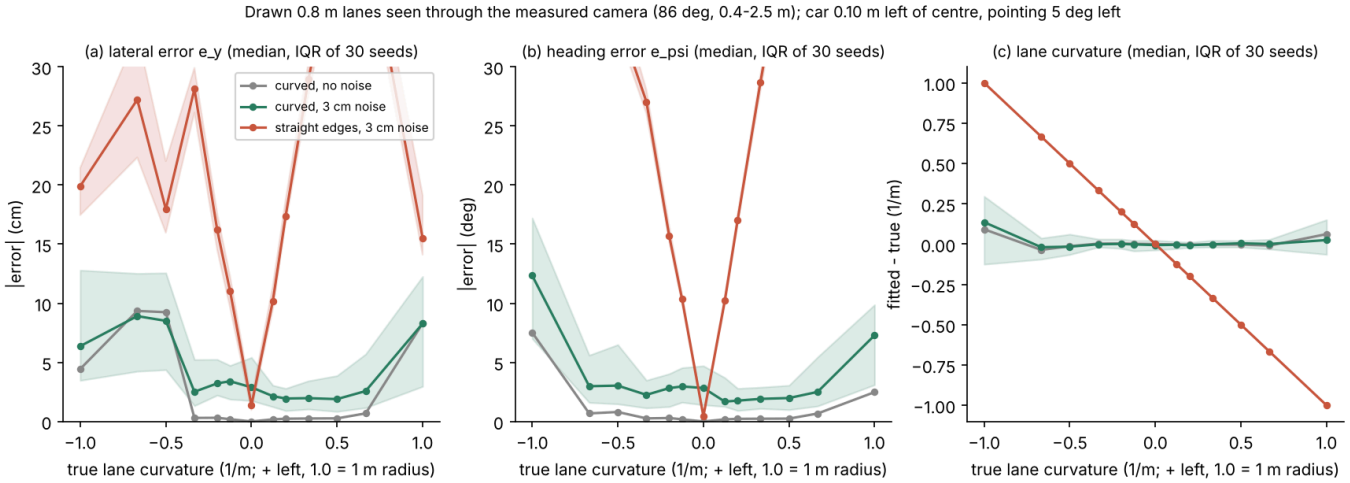
Steps of `LaneDetector.detect` (top) and the model for a lane curving left (bottom).

- Both edges are concentric arcs about one reference arc (heading ψ, curvature κ) at offsets d_L, d_R; signed distance of cone j, with $s_j = x_j\cos\psi + y_j\sin\psi$, $\ell_j = -x_j\sin\psi + y_j\cos\psi$:

$$D_j = \frac{2\ell_j - \kappa(s_j^2 + \ell_j^2)}{1 + \sqrt{1 - \kappa\,(2\ell_j - \kappa(s_j^2 + \ell_j^2))}}, \qquad \min_\theta \sum_j \rho\big(D_j - d_{edge(j)}\big) + \rho\big((\sigma/\kappa_{typ})\,\kappa\big)$$

- θ = (ψ, κ, d_L, d_R); soft-L1 loss ρ (0.05 m); curvature prior σ = 0.05 m, κ_typ = 0.5 1/m. Cones > 0.15 m off their edge are dropped and refitted (≤ 2 times).
- Outputs $w = d_L - d_R$, $e_y = -(d_L + d_R)/2$, $e_\psi = -\psi$; `NO_LANE` unless 0.3 ≤ w ≤ 3.0 m.
- **AUTO + width memory:** the edge further left is the left edge, whatever its colour; $w_{mem} \leftarrow w_{mem} + 0.2(w - w_{mem})$, so one edge still gives the centre.
- **Camera pan:** the TF assumes zero pan, so a camera turned by δ gives $e_{\psi,meas} = e_{\psi,true} + \delta$ → hold the pan at 0.

S3: is fitting the edges together better? How accurate is it? 3 cm noise, 200 seeds: with a 3-cone row, 0.69° vs 3.03° heading RMS (equal at 12 cones). Curved mode: e_y 1.9–3.4 cm for radius ≥ 1.5 m (R1b allows 10 cm).



Drawn 0.8 m lanes, 30 seeds: straight mode (red) fails on any curve; curved mode (green) stays low until the tightest bends → straight for Stage 1, curved for curved roads.

- **Simulator (27 Oct):** real `MovementPolicy` + car 20's maps, camera 86°, 0.4–2.5 m, 3 cm noise, 0.2 s. 76/90 runs pass on six tracks, no contacts; junctions least robust (17/25 with 20% missed cones). Uncorrected yaw

±4° fails a whole turn direction → **the pan hold is a precondition for turns**. Camera optimistic (20 vs 14 real cones per step) → pass rates are an upper bound.

S4: why did the lane look turned on real drives? (recordings, 6 Oct) Steering was 0, yet every drive read e_ψ = +6° to +11°. A real heading would make the car drift; rotated cones wouldn't.

Drive	Median e_ψ	Drift if real	Measured drift
straight_even	+9.6°	+0.16 m	−0.05 m
messy_cone	+10.9°	+0.19 m	−0.03 m
outlier_cone	+8.0°	+0.38 m	−0.01 m
long_track	+6.5°	+0.32 m	−0.07 m

→ the camera was turned, not the car (pan never commanded). **Pan held at 0 from 10 Oct.** Same drives: width 0.74–0.84 m, edge RMS 2.8–4.5 cm, lane seen to ≈ 2 m → cone range cut to 2.5 m. **Limits:** one arc can't fit an S-bend; a wrong-colour cone in a short row can stop the car; no ground-truth e_y on the car.

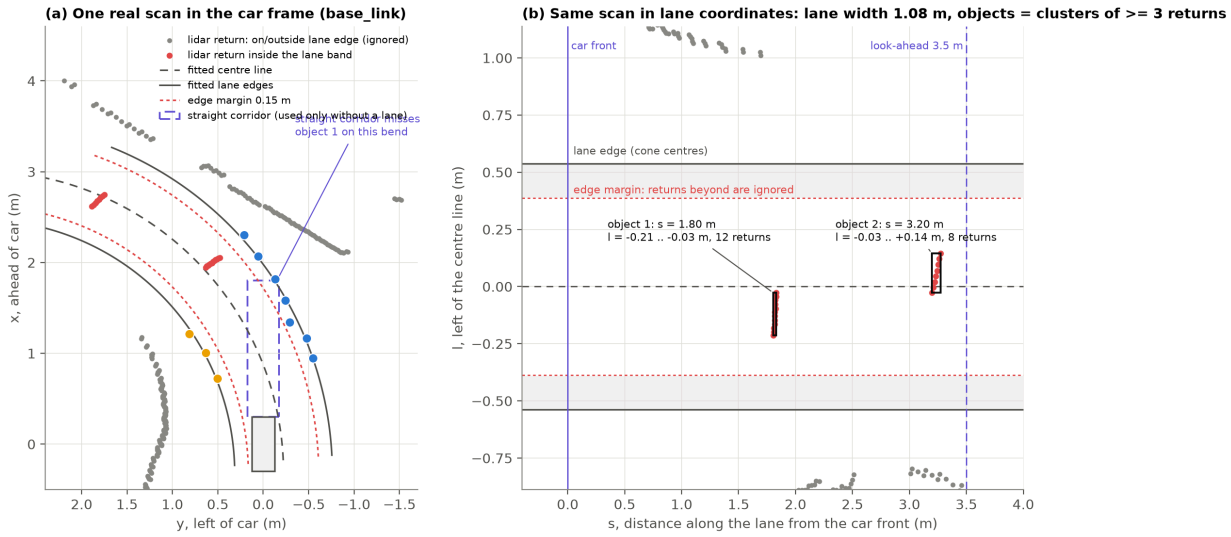
2.3 Zhenling Zhang — cone filter, obstacles, avoidance and local map

Built: cone filter, lidar obstacle detection in lane coordinates, obstacle stop and brake, avoidance planner, local map, closed-loop obstacle simulation. **PRs** #11 #19 #21 #28 #33 #34. All five ran on cars; the 26 Oct drive is the best video.

S2.

- **Cone filter:** keep $0 < x$, $|y| \le 2.0$ m, range ≤ 3.5 m, $-0.15 \le z \le 0.25$ m, confidence ≥ 0.5; merge same-colour cones < 0.10 m apart.
  - *Why range ≤ 3.5 m:* the lane fit describes each edge with one constant-curvature arc, so it can only follow the lane a short way ahead. Far cones from the next bend, such as the second half of an S, would pull the fit near the car the wrong way. A much shorter range leaves too few cones on each edge, and the fit jumps from step to step.
- **Obstacle detection:** lidar points arrive in `base_link` as (x, y) and are moved into lane coordinates on the fitted centre arc (radius $R = 1/\kappa$; both edges are concentric with it): $s = R\theta - 0.30$ m is the distance along the arc from the car's front, and $l = R - \rho$ is the offset across it, where θ is the angle swept about the arc's centre and ρ the point's distance from that centre (left bend; signs flip for a right bend). Both edges then lie at $l = \pm w/2$ for every s, so a bend becomes a straight strip.
  - *Range:* only points with $0 < s \le 3.5$ m count.
  - *Inside the lane:* $|l| < w/2 - 0.15$ m, so returns from the cones are ignored.
  - *Objects:* neighbouring returns less than 0.15 m apart form one object, which needs at least 3 returns.

Fig. Z4 Obstacle detection on real data (drive 22 Oct, 17-59-26, step 4): the lidar also hits the cones, so only returns inside the lane band count



Real scan, 22 Oct. (a) Car frame: the lidar also hits the cones, and a straight corridor misses object 1 on this bend. (b) Lane coordinates: the edges are straight lines; returns within 3.5 m and inside the edge margin form two objects.

- **Obstacle stop:** seen 3 steps in a row, within

$$d_{stop} = (t_b + t_{scan})\,\max(v_{target}, v_{measured}) + m, \qquad t_b = 0.8\ \text{s},\ m = 0.2\ \text{m}\ \ (0.70\ \text{m at}\ 0.5\ \text{m/s})$$

Term	Value	Meaning
$t_{scan}\,v$	t_{scan} = each scan's own age	Distance travelled since the scan was taken
$t_b\,v$	t_b = 0.8 s per m/s	Braking travel. Chosen in simulation: no hit for true braking up to 1.0 s per m/s
$\max(v_{target}, v_{measured})$	larger of the two	Covers a car faster than its target, and a car still speeding up
m	0.2 m	Gap left between the car's front and the obstacle after stopping

- **Avoidance planner:** plans one avoidance path past all known obstacles, as an offset $d(s)$ from the lane centre in lane coordinates, re-planned every step. The MPC follows it.

1. Path structure. The path is a chain of segments: straight **holds** at a fixed offset (beside an obstacle, or on the usual line) and **quintic moves** between two offsets. Obstacles far apart get a hold beside each; for close ones, one move runs straight into the next.

2. Quintic move. A move over $[s_i, s_i + L]$, with $t = s - s_i \in [0, L]$, is

$$d(t) = c_0 + c_1 t + c_2 t^2 + c_3 t^3 + c_4 t^4 + c_5 t^5$$

It leaves the start state (d_0, d_0', d_0'') and reaches the end state (d_1, d_1', d_1''): offset, slope and curvature. These six conditions fix the six coefficients. The three at $t = 0$ give

$$c_0 = d_0, \qquad c_1 = d_0', \qquad c_2 = \tfrac{1}{2}\,d_0''$$

With what is left to reach at $t = L$ after those terms,

$$\Delta d = d_1 - (c_0 + c_1 L + c_2 L^2), \qquad \Delta v = d_1' - (c_1 + 2c_2 L), \qquad \Delta a = d_1'' - 2c_2$$

the three at $t = L$ give

$$c_3 = \frac{10\,\Delta d - 4\,\Delta v\,L + \frac{1}{2}\Delta a\,L^2}{L^3}, \qquad c_4 = \frac{-15\,\Delta d + 7\,\Delta v\,L - \Delta a\,L^2}{L^4}, \qquad c_5 = \frac{6\,\Delta d - 3\,\Delta v\,L + \frac{1}{2}\Delta a\,L^2}{L^5}$$

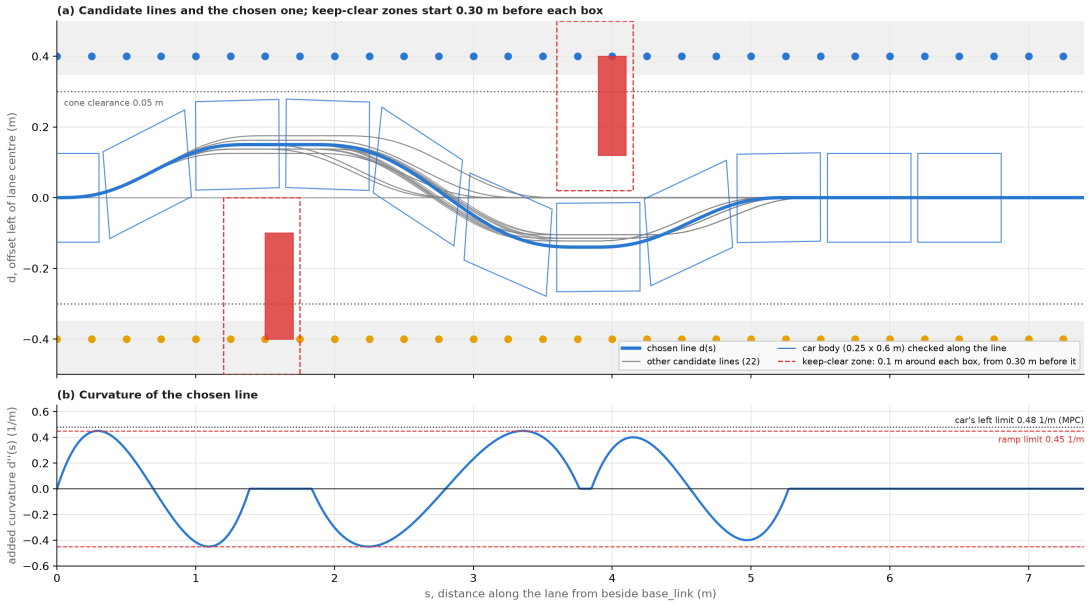
So the coefficients depend on both end states and on L, and offset, slope and curvature are continuous at every join. Every move ends on a hold ($d_1' = d_1'' = 0$). Only the first move of a step may start mid-turn, from where the previous step's path is now; the others start on a hold too.

Move length. For a move between two holds with offset change $D = d_1 - d_0$, the peak curvature is $\frac{10}{\sqrt{3}}\frac{|D|}{L^2}$, so the shortest move within the curvature limit is $L_{min} = \sqrt{\frac{10}{\sqrt{3}}\frac{|D|}{\kappa_{max}}}$. A move that starts mid-turn first tries to end as the car's front reaches the obstacle's stretch, then the lengths $L_{min} \times \{1, 1.25, 1.5, 2, 3\}$, and takes the first whose peak curvature stays within κ_{max}.

3. Choosing the combination.

- **Joining:** before the first group, hold the current line, then take the shortest move, arriving as the car's front reaches the stretch. Between groups, one of four ways: keep the offset; a move centred between them; a move spread over the whole gap; or back to the usual line and out again. After the last group, one move back to the usual line at the earliest point where the body keeps its clearances.
- **Check and search:** each candidate is checked every 0.05 m with the car's body, a 0.25 × 0.60 m rectangle turned along the path. The search goes group by group, keeping the 6 cheapest partial paths, for up to 3 groups.

- **Ranking**, in order: obstacles passed (nearest first) → remaining distance to passing the next → already beside the obstacles before d_{stop}



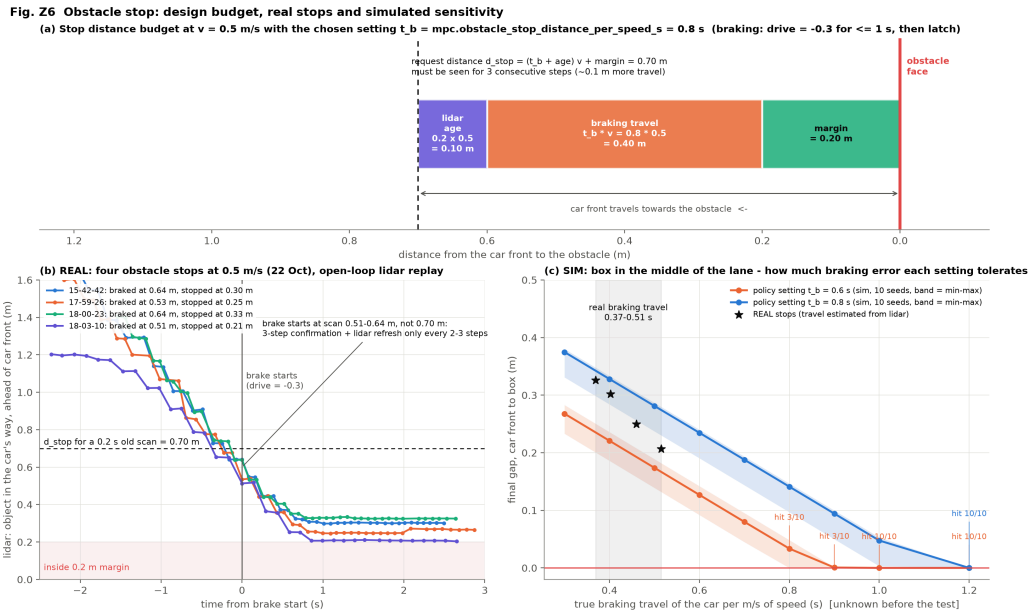
Simulation, 0.8 m lane, box right then left. (a) Grey: 22 candidates; blue: chosen line, body checked along it. (b) Curvature continuous, inside 0.45 1/m (car's left limit 0.48).

- **Local map:** dead-reckoned pose, nearest-cone matching (0.12 m), half a rigid fit per step; recalled cones only added once the batch already gives a lane.

S3: retune against needless stops and hits (21 Oct). Closed loop, 10 seeds per case, braking 0.6 s per m/s. Narrower corridor (0.175 m), wider edge margin (0.15 m), 3 confirmations, braking travel in d_{stop}:

Case	Previous	Retuned
Box right, 0.5 m gap	3/10 (7 needless stops)	10/10
Box in the middle (must stop)	**0/10: hit every time**	10/10, 0.19 m short
Slalom, 2 boxes 2.5 m apart	3/10	10/10

S4: braking measured from the lidar (22, 26 Oct). Correcting for scan age, $t_b = \big((d_{scan} - v\,t_{scan}) - d_{final}\big)/v$.

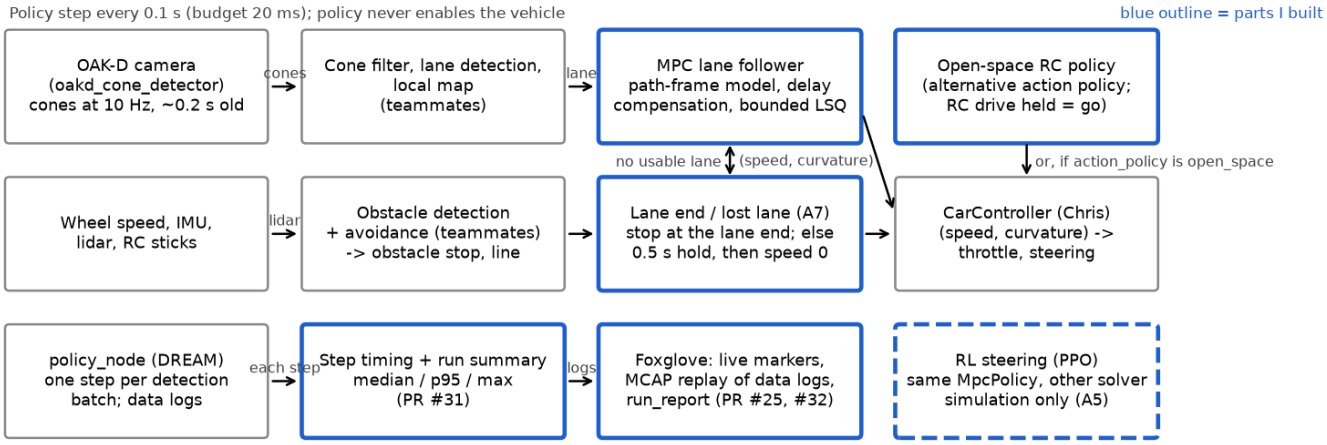


(a) Stop distance at 0.5 m/s = scan age + braking + margin = 0.70 m. (b) 5 real stops: braked at 0.51–0.64 m readings, stopped 0.21–0.33 m short. (c) Simulated tolerance per setting; stars are the real stops.

- Real t_b 0.37–0.53 s < the cautious 0.8 s → always kept 0.21–0.33 m. Odometry claimed 1.1 m of braking; the lidar showed 0.32 m (wheel speed holds its last value).
- **26 Oct (best video):** passed a box right, then left (1.4 m apart), then stopped for a middle box whose gaps (0.31, 0.12 m) were under the 0.40 m needed: braked at 0.69 m, stopped 0.23 m short.
- S-bend (20 Oct): curvature flipped sign in 18/28 steps → 0/28 with the 3.5 m range. Local map halved the p95 lane jump in all four 22 Oct drives. **Limits:** static obstacles only.

Part 3: Movement policy (MPC)

3.1 Shared: the MPC all three members build on



Lane (2.2) in, (speed, curvature) to car control (4.1) out, once per cone batch within 20 ms. Markers (3.3) and the turner (3.3, 3.4) switch what the MPC tracks.

Model (Peiyu). Lateral error e, heading error ψ, curvature κ; the car moves $d = v\Delta t$ per step, Δt = 0.1 s:

$$e_{k+1} = e_k + d\,\psi_k + \tfrac{d^2}{2}(\kappa_k - \kappa_k^{path}), \qquad \psi_{k+1} = \psi_k + d\,(\kappa_k - \kappa_k^{path})$$

$$\min_{\kappa_0..\kappa_{N-1}} \sum_k q_e\,(e_k - e_k^{line})^2 + q_\psi\,\psi_k^2 + q_\kappa(\kappa_k - \kappa_k^{path})^2 + q_\Delta(\kappa_k - \kappa_{k-1})^2 \quad \text{s.t.} \quad \kappa_{min} \le \kappa_k \le \kappa_{max}$$

- **Delay compensation:** predict forward by $d_0 = v \cdot \text{age}$ (≈ 0.2 s) with the previous command; send only the first curvature.
- **Bounded least squares.** The fixed-speed MPC precomputes M in $\|Mu - c\|^2$. With measured wheel speed, the MPC rebuilds the speed-dependent terms of M at each solve and reuses the fixed weights and curvature penalties. The $4N$ cap in the BVLS solver limits iterations. R6 uses measured step time.

Setting	Stage 1 design (3.2)	Car 20, `dev` (3.3)
N	20 (1 m at 0.5 m/s)	12
$q_e/q_\psi/q_\kappa/q_\Delta$	3 / 1 / 0.1 / 3	10 / 1 / 0.1 / 2
κ bounds	±3.0 1/m	−0.92 to +0.48 1/m (measured, 4.1)
e^{line}	0 (centre)	lane side offset (3.3)
v in d	target	measured wheel speed (3.4)

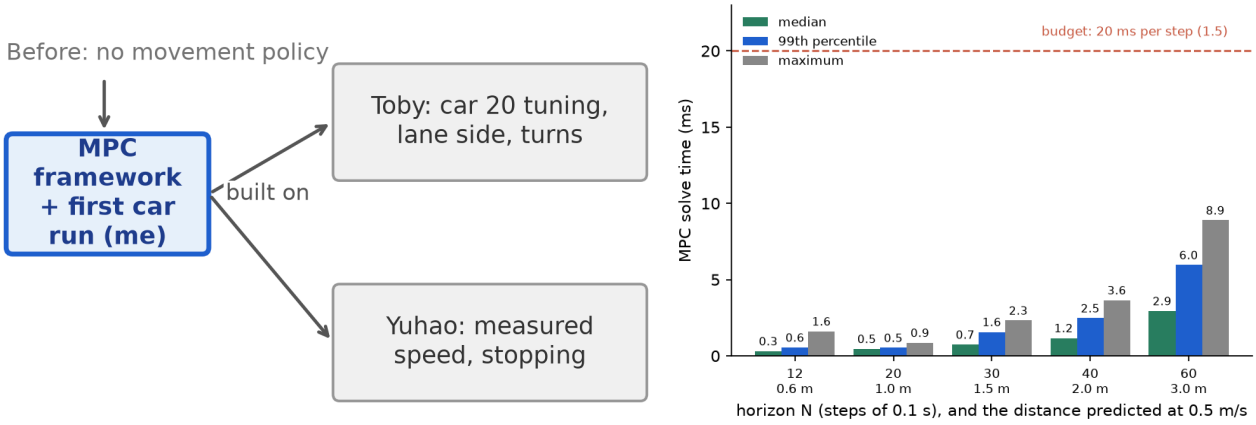
Constraint	Handled as	Why
Curvature limit	hard bound	the real steering limit; bounds always feasible for `bvls`
Steering rate	cost q_Δ	a rate limit needs a general QP; the cost keeps R4 at 0.16
Lane edges	left out	a hard state limit is infeasible when noise puts the car near an edge
Speed	not a variable	would make the model bilinear; car control holds speed

3.2 Peiyu Wang — MPC base, MPC vs RL, virtual debug, keeping the code and cars working

Built: the MPC framework and first car run, the Stage 1 design (pure pursuit, MPC grid, RL, milestone M1), lane-end and lost-lane stops, car models, the Foxglove replay tool, car bring-up, merges and reviews. **PRs** #1 #8 #16 #25; reviewed and merged #27.

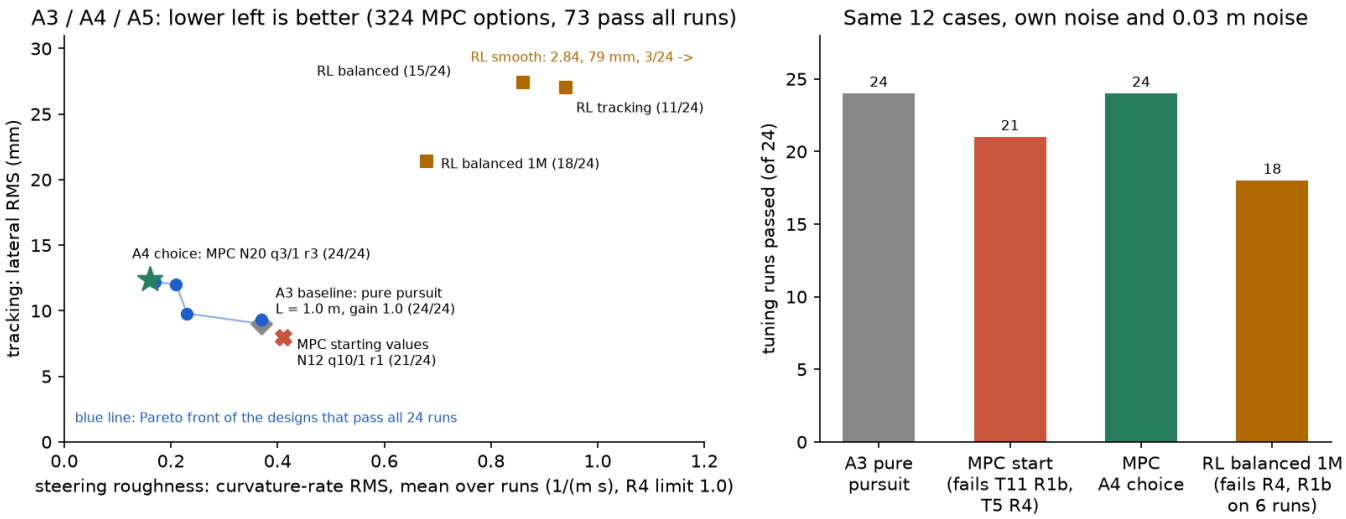
1. The MPC base (S2, S4). *Before:* no movement policy. I wrote the MPC framework (#1: the model and solver in 3.1) and the first car run (#8). The team built on it: Toby tuned it for car 20 and added lane side and turns (3.3), Yuhao added measured speed and stopping (3.4).

- **Lane end and lost lane (S2):** no lane and the farthest cone ≤ 1.5 m ahead → the rows ended: follow the last plan, request the stop at roll-out distance (`stop_distance_per_speed_s` × v) from 0.25 m past that cone. No lane otherwise → half speed ≤ 0.5 s, then a latched stop (R7: 0.6 s with the 0.2 s age). *Before:* the car stopped 0.38–0.91 m short (the last pair leaves the 80° view ≈ 0.6 m early, so the lane looked lost); *after:* 0.21–0.24 m past (R5).
- **Time budget (S4, car 20, 6 Oct):** batches every 0.1 s, latency 0.197 s → 20 ms per step allowed. Solve N = 20: 0.46 ms median, 0.53 ms p99; N = 60 still 5.96 ms → the horizon is limited by camera range, not CPU.



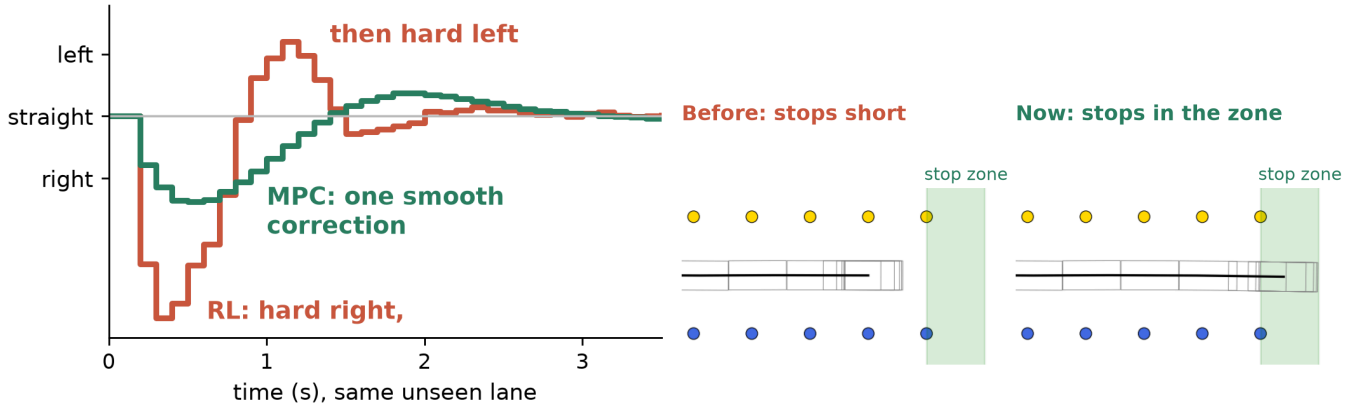
Left: the MPC base and who built on it (blue: mine). Right: MPC solve time on car 20's computer, 300 solves per horizon, against the 20 ms budget.

2. Deciding MPC or RL (S3). *Why decide:* a learned policy might track better, but it had to stay smooth on lanes it was never tuned on. *How:* every design ran in the same harness with the same cone filter and lane-end rule; only the steering differed. First the MPC settings: 324 settings × 24 runs (12 cases, ± 0.03 m noise).



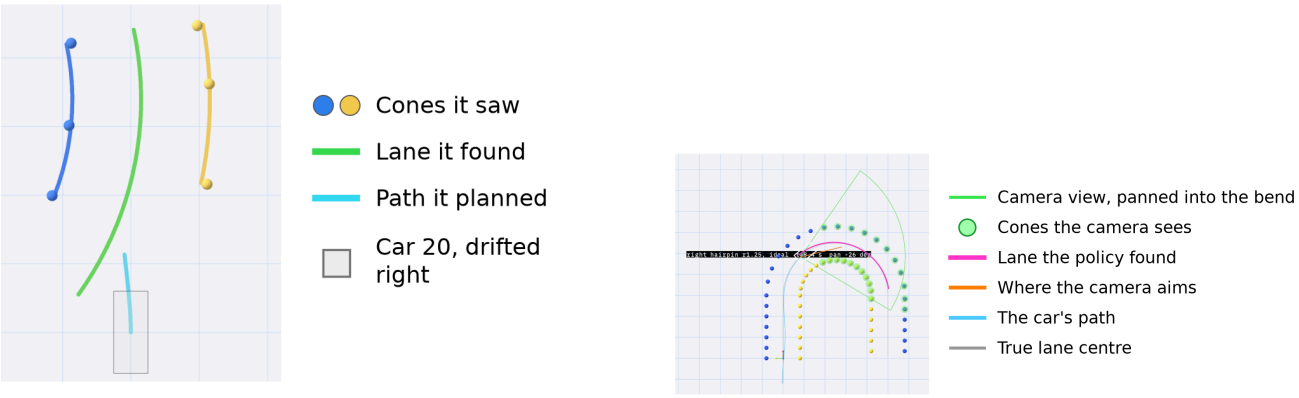
Tracking against roughness; Pareto front of passing settings; star = chosen (N 20, 3/1/0.1/3): the smoothest passing. Pure pursuit tracks 3 mm better but steers 2.3× rougher.

RL: PPO replaces only the solver; bounded penalties $P(x, s) = 1 - e^{-(x/s)^2}$, reward $r = 1 - aP(y, 0.1) - bP(\psi, 0.1) - cP(\dot{\kappa}, 1.0)$. Its first reward was exploited (unbounded penalties made leaving the lane best); with bounded penalties it drove the lane but still weaved. **M1 (unseen cases U1–U3, opened once, rule fixed before):** RL failed R4 on U1 (1.51) and U2 (1.59); the MPC passed all (U2: 0.51 vs 0.80) → ***MPC chosen***, because it stayed smooth on lanes it was never tuned on.



Left: the steering each design asks for on U2 (rerun at `47f8838` with `rl_models/balanced_1M.zip`; data in `peiyu/figures/m1_u2_steering.json`). RL turns hard right, then hard left; the MPC makes one smooth correction. Right: the end of T1 in the simulator's own top view (`stage1_scenarios.plot_result`), before the lane-end rule (`01a47a0`, 0.50 m short, FAIL) and with it (`9a7f9d8`, 0.20 m past, PASS; neither with the 0.2 s age).

3. Virtual debug (S4, S5). *Before:* when a car run went wrong, the team had the car's path and text logs: watch where it drove, then guess why. *Built:* every step's observations and actions are logged; `scripts/mcap_export.py` replays a data log through the policy into Foxglove, step by step, with the cones the car saw, the lane it found and the MPC's plan, and the car publishes the same drawings live (#25). *After:* problems are found by replaying runs instead of rerunning the car; Toby's turn checks read `/plan/intersection` in Foxglove, and Long's on-car live viewer draws the same markers (their full sections). A replay runs the current code, so car runs are judged by their logged actions (on the run below, the replay chose the car's own actions on every step). The camera pan prototype (`230c75b`, S3) used the same export: turning the camera into tight bends keeps the lane in view only when the camera's angle at each image is known; with the latest command instead, it lost an oval the fixed camera finished, so the pan stays at 0.



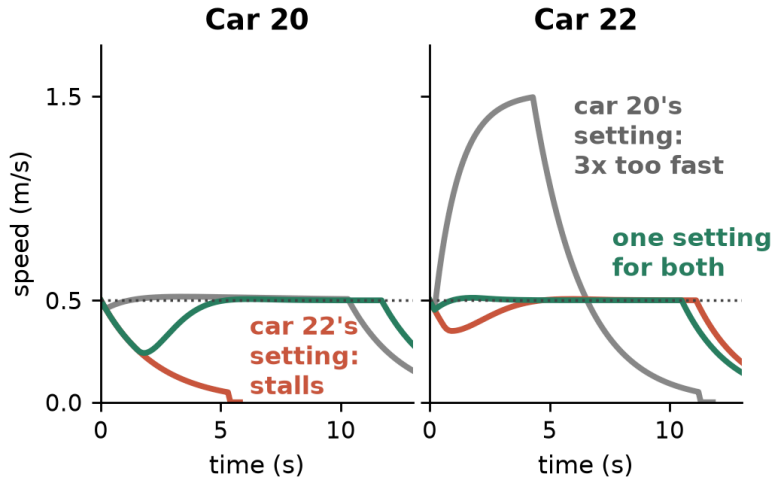
Left: a car 20 run replayed in Foxglove: the car has drifted right and the MPC's plan steers it back. Right: the camera pan prototype in Foxglove (hairpin, ideal pan): the camera's view turned into the bend and the lane the policy found.

4. Keeping the code and cars working (S1, S4).

- **Merge conflicts:** eight people pushed to `dev` at once. Chris's controller (#17), Toby's calibration (#15), the first car run (#8) and the Stage 1 design (#16) landed together; I resolved six merges (largest 22 conflict hunks, `fb2442b`): `dev`'s shared code kept, the MPC features ported onto it, tests re-run; lab weights 10/1/0.1/2 taken, since A4's with #15's change weight left a 3 m radius curve.
- **Code review (#27, Toby's lane side):** found and fixed a duplicate setting, a reset that kept the last run's lane side, a first step planning from the previous run's line, a missing test with avoidance and a stale docstring;

merged `dev` in, re-ran the wide-track results (CI 560 passed), then merged.

- **Every car differs:** with car 20's values car 22 drove 1.4 m/s for 0.5; car 22's values stall the car 20 model → car models and one setting checked on every model (#18, waiting for review). Car 22 stopped 0.8 s after losing the lane; its steering stayed biased (unsolved).
- **Cars that would not start:** car sessions kept starting with a car that would not run, each time for a different reason, and each cost restarts, log reading and redeploying the policy: the cone detector crashed under OpenCV 5 (`KeyError 16`) → `publish_rgb_image: false` (#24); the OAK-D dropped off USB → restart only the detector; DREAM's `runtime_runner` processes at 70–80% CPU each (reported upstream); the policy running but the car not moving (wrong `left_colour`, stray cones → moved the cones); a flat motor battery (drive applied, wheel speed 0 → check the battery first). Car 20 was shared, so every check was read-only first and its source was backed up before deploying.

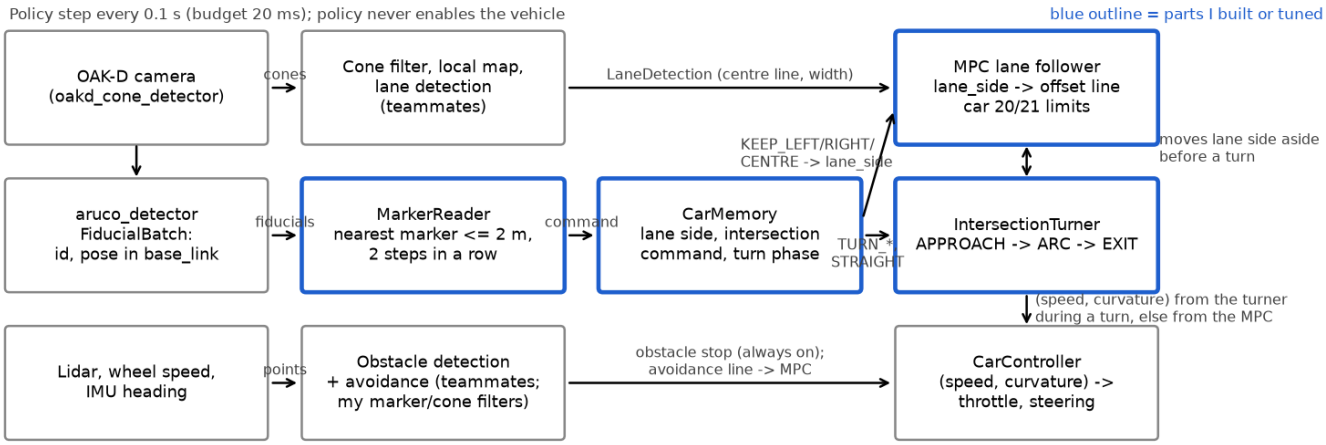


Speed on T1 at 0.5 m/s through `CarController` and each measured car model (`45eb015`, data in `peiyu/figures/car_models_speed.json`): car 22's values stall car 20; car 20's drive car 22 3× too fast; one setting (#18) holds both.

3.3 Toby Lee — test harness, MPC tuning, lane side, marker actions and intersection turns

Built: Stage 1 test cases and harness (#6 #10), car 20 MPC settings and lane offset (#15), lane side (#27 #30), marker actions and the intersection turner (#35, with Yuhao), the turn simulation suite and the left-turn fix (`fix/left-turn-steering-and-obstacles`, **not merged**). Car sessions on cars 20, 21, 13 and 14.

Where markers and turns fit in the policy (S1). Markers decide *which line* the MPC follows; the turner takes over only for the arc.



Blue = my parts. In: lane (2.2), ArUco poses, lidar. Out: a lane side for the MPC (3.1), or (speed, curvature) from the turner during a turn, to car control (4.1). Constraint from 4.1: car 20 steers +0.48 1/m left, −0.92 right.

- **Requirements I tested against:** R1–R7 (0.1) for lane following; for turns, leave by the commanded road without touching a cone; otherwise stop.
- **Assumptions:** cones mark a lane 0.8–1.0 m wide; the marker pose is in `base_link`; the camera points straight ahead (pan held, 2.2). A wrong camera yaw or steering reach breaks turns first (S3).

How the car picks its line and plans a turn (S2): test harness, lane side, markers, arc-start window.

- **Test harness** (`stage1_scenarios.py`). 12 tuning cases T1–T12 (start offset and heading, 0.8–1.2 m lanes, noise, missed or extra cones, delay, a heavier car, a camera blackout); U1–U3 held back. Any `run_policy()` design (MPC, RL, pure pursuit) drives dream-gym closed loop and is scored on R1–R7.
- **Lane side** (`policy.py`). The MPC tracks a line offset o from the centre. With edge clearance c = 0.28 m:

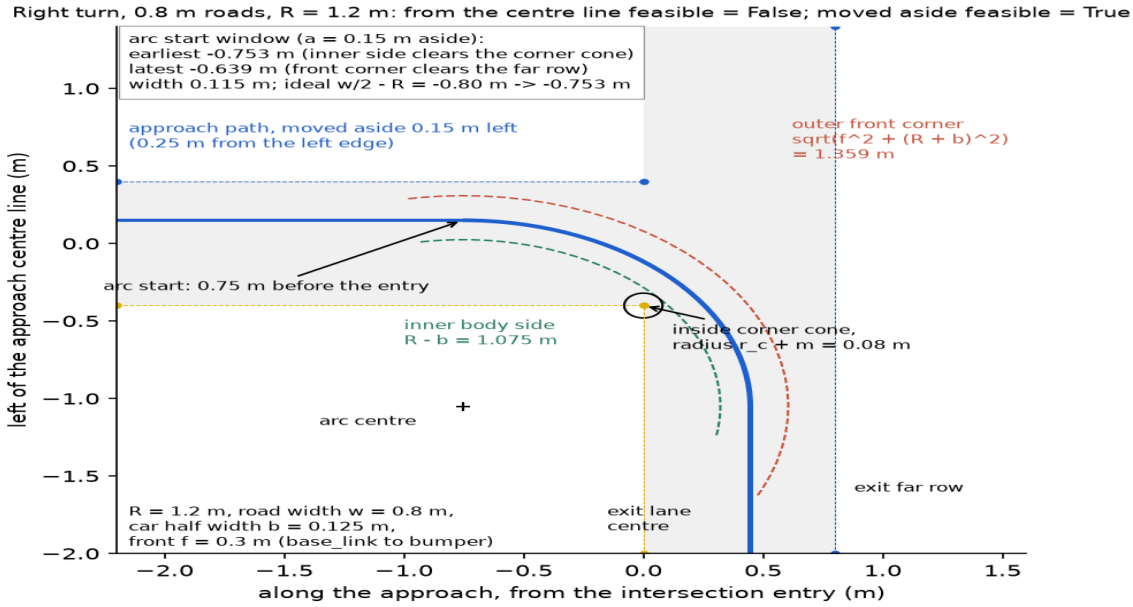
$$|o| \le \tfrac{w}{2} - c, \qquad o^* = \pm(\tfrac{w}{2} - c) \text{ or } 0, \qquad o_k = o_{k-1} + \mathrm{clip}\big(o^* - o_{k-1},\ \pm 0.3\,\Delta s_k\big)$$

±0.12 m in a 0.8 m lane, ±0.22 m in a 1 m lane. The MPC sees the offset line as its path: $e' = e - o$ and $\kappa' = \kappa/(1 - \kappa o)$ for the offset line.

- **Markers → actions** (`markers.py`). Nearest posed marker ahead within 2 m, fired after 2 steps in a row. Keep-side IDs set the lane side; turn and straight IDs start the turner; an unknown ID latches a stop.
- **Turner: APPROACH → ARC → EXIT** (`intersection.py`). Move to the outside line, then drive an arc of radius $R = \max(R_{cfg}, 1/\kappa_{max})$ by pure pursuit, $\kappa = 2y/(x^2 + y^2)$. R_{cfg}: preferred radius in the config (1.1 m left, 1.2 m right); κ_{max}: the car's steering limit that way (car 20: 0.48 left, 0.92 right 1/m), so the radius is never tighter than the car can drive; (x, y): the pursuit target a lookahead ahead on the path (0.5 m on `dev`, 0.8 m on the fix branch), in the car's frame. Hand back to the MPC once the lane is found again after ≥ 60°. The arc may only start where the body clears both corners:

$$\underbrace{-\sqrt{(R - b - r_c - m)^2 - (R - a - w/2)^2}}_{\text{inner side clears the inside corner cone}} \le s \le \underbrace{w_{cross} - r_c - m - \sqrt{f^2 + (R + b)^2}}_{\text{outer front corner clears the far row}}$$

s: arc start along the approach (m, 0 = entry); a: how far the car moved aside; w, w_{cross}: approach and crossing road widths; b = 0.125 m half width; f = 0.30 m to the front; $r_c + m$ = 0.08 m cone plus margin. The code also corrects for heading error (Yuhao, 3.4). No valid s → braking stop (the "geometry stop").



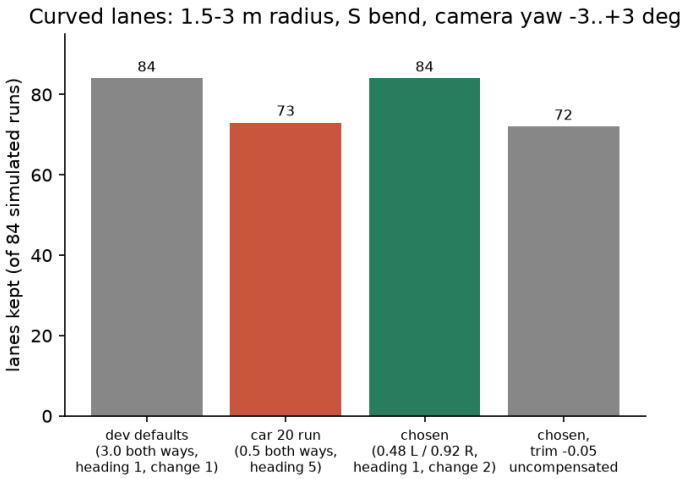
Window width	0.8 m roads	1.0 m roads
Right, R = 1.2 m, from the centre / moved aside	none / 0.12 m	— / 0.45 m
Left, R = 2.08 m (0.48 1/m, car 20)	none	0.014 m

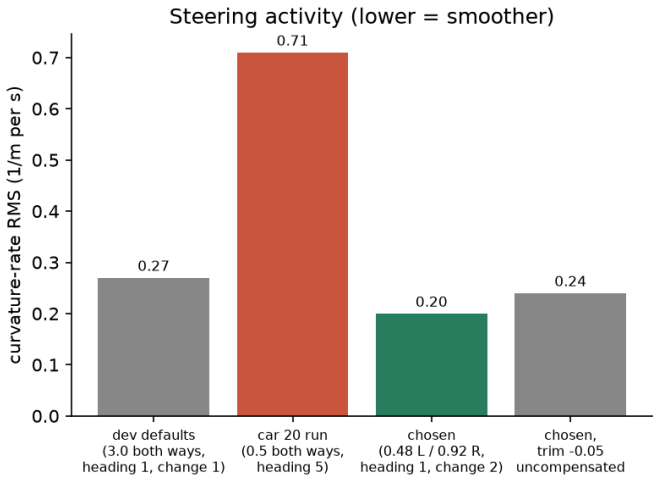
Window width	0.8 m roads	1.0 m roads
Left, R = 1.1 m (0.91 1/m, the fix)	0.17 m	0.48 m

Moving aside widens the window; a large radius closes it. This is why left turns failed (S3).

Simulation: tuning the MPC for car 20, lane side, and why left turns failed (S3).

- **Which MPC settings suit car 20? (12 Oct).** Settings for Peiyu's MPC, not a new controller. 84 curved-lane runs per setting (straight, 1.5–3 m radius curves, S-bend × 3 camera yaws × 4 seeds), 0.8 m lane, 0.2 s delay.

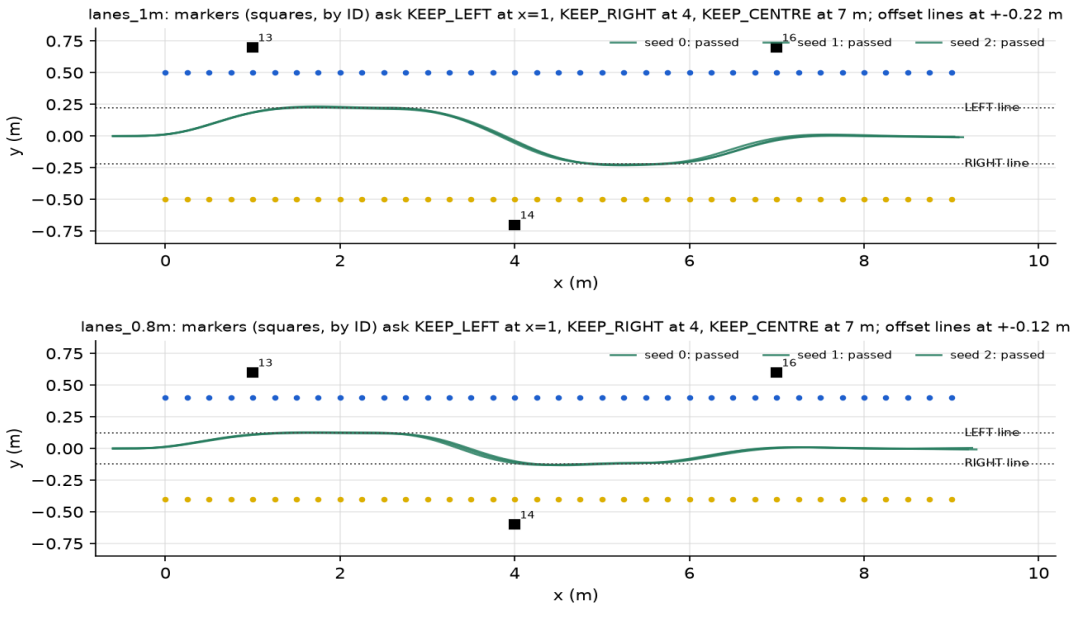




Changed	From → to	Why
Curvature limits	±0.5 → +0.48 / −0.92 1/m	Chris's measured reach; a 0.48 right limit kept 0/12 of the 1.5 m right curves, 0.92 kept 12/12
Heading weight q_ψ	5 → 1	5 lined the car up with the lane but let it drift off centre
Change weight q_Δ	1 → 2	steering 0.27 → 0.20 1/(m s); 3 was smoother but ended 0.24 m off centre vs 0.18

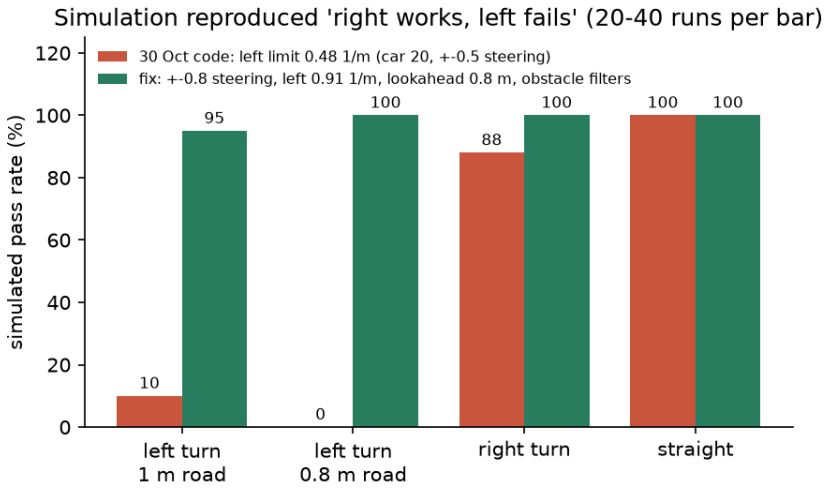
Result: 73 → 84 of 84 lanes kept, steering 0.71 → 0.20 1/(m s); 150 straight tracks 97% (91% `dev`). **Limit:** those 150 tracks were also used to choose, so 97% is not held-out. Nothing is learned; only settings are chosen.

- **Lane side (26 Oct).** c = 0.40 m left no room to move in a 0.8 m lane → 0.28 m: 40/40, side error ≤ 0.017 m.



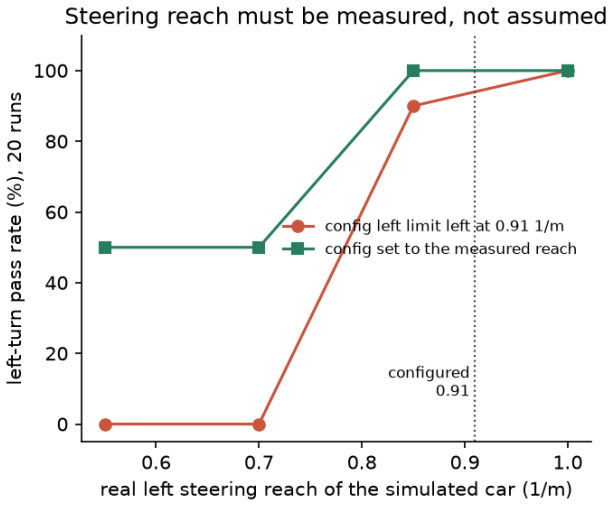
Keep left, right, then centre. Marker IDs 13, 14, 16 are the simulation's mapping, not `dev`'s.

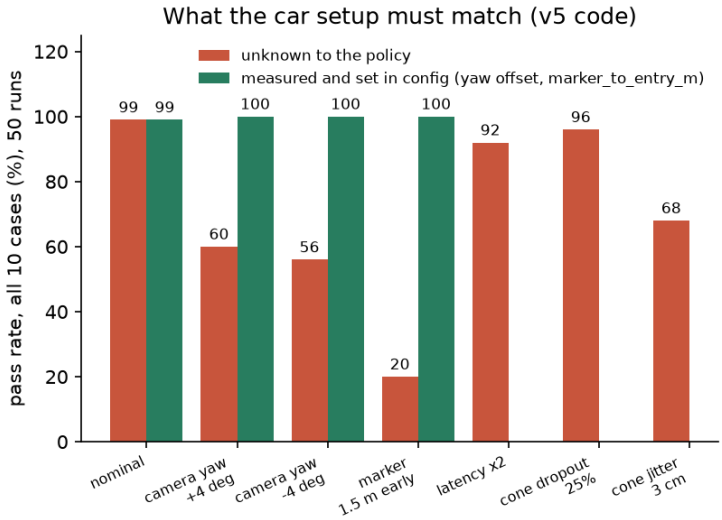
- **Why do left turns fail on car 20? (1–2 Nov).** `marker_simulation.py` runs the whole policy closed loop through left, right and straight crossings (0.8–1.5 m roads, noise seeds); `--set` sweeps one setting at a time.



Reproduces the car: left 2/40, right 35/40. Car evidence: Yuhao's 30 Oct geometry stop (3.4).

Fix (branch, not merged)	Problem	Result (sim)
Steering ±0.5 → ±0.8 (Chris) and left limit 0.48 → 0.91 1/m	arc too wide for the crossing	left 39/40
Pure pursuit lookahead 0.5 → 0.8 m	sharper steering swung the move-aside	43/48 vs 33/48
Ignore lidar near a seen marker or a known cone while turning	stands and cones read as obstacles	phantom avoidance 29% → 4%; turns 40% → 68%



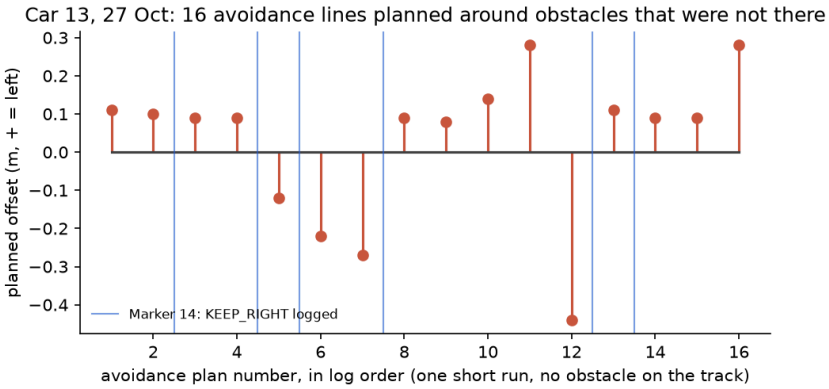


Left: if the real left reach is 0.7 1/m or less, every left turn fails, so 0.91 (not ground-measured, 4.2) must be measured before use. Right: camera yaw and marker position must be measured; latency and dropout matter less.

Car tests: does each link from marker to action work? (S4)

Question (car, date)	Method	Result → decision
How far is sim from the lab? (4 drives, 12 Oct)	recorded lane width, cone noise, delay, range	0.77 m, 0.040 m, 0.21 s, 2.54 m → sim uses 0.8 m, 0.03 m, 0.2 s, 2.5 m
Does lane side work? (20, 20 Oct)	only `lane_side` changed to LEFT	kept left, by eye only (no log)
Do markers reach the policy? (9 runs)	count marker batches	0 → start with `car_management.sh --aruco`

Question (car, date)	Method	Result → decision
Is the command right? (13, 27 Oct)	log the command	"Marker 14: KEEP_RIGHT", 6 times
Full turn (2 Nov)	—	not shown: wrong branch, then no `--aruco`



Car 13, no box on the track: marker stands planned 16 avoidance lines. This led to the lidar filters in S3. ***Limit:*** *they hide a real object within 0.15 m of a marker or 0.12 m of a cone.*

3.4 Yuhao Liu — measured-speed MPC, cone memory and junction planning

Built measured-speed prediction and solver-failure stops (#14), an offline final-pair estimator, a cone-memory U-turn and an experimental cross/T-junction navigator. In Toby's default turner, added grouped marker IDs, earlier turn preparation, heading-aware arc admission and a persistent stop latch (#35, #36). Diagnosed car 20's left-turn failure.

S2. Measured wheel speed sets the prediction step distance $d = v\Delta t$ and the delay projection $d_\tau = v\tau$. A failed solve requests a stop. The default turner starts preparation on the outside of the lane after confirmation and checks an arc tangent to the actual heading. At the 0.48 1/m left limit, the radius is 2.083 m. A rejected turn keeps its stop latched after the turn object clears.

Calibrated MPC integration. I integrated Toby's asymmetric car 20 steering limits and lane offsets with my measured-speed prediction and stopping logic in PR #15. In PR #16 I resolved the follow-on CarController merge and added checks that reject non-finite control inputs. Calibration repair · CarController integration

- **Cone memory.** Wheel and gyro odometry reproject stored cones as the car moves. The U-turn controller holds its committed path through short camera dropouts and recomputes steering from the current pose.
- **Junction path finding.** The navigator queues the ArUco command and uses cone rows and current lidar to verify the requested exit. An unverified exit allows a finite approach ending before the junction. A closing row blocks commitment. The selected centre path feeds MPC until fresh exit geometry and heading alignment allow lane following.

S3: does measured speed help? 3 scenarios × 2 inputs × 5 seeds, 0.2 s delay; distance-weighted $e_{RMS} = \sqrt{L^{-1}\int_0^L e(s)^2 ds}$:

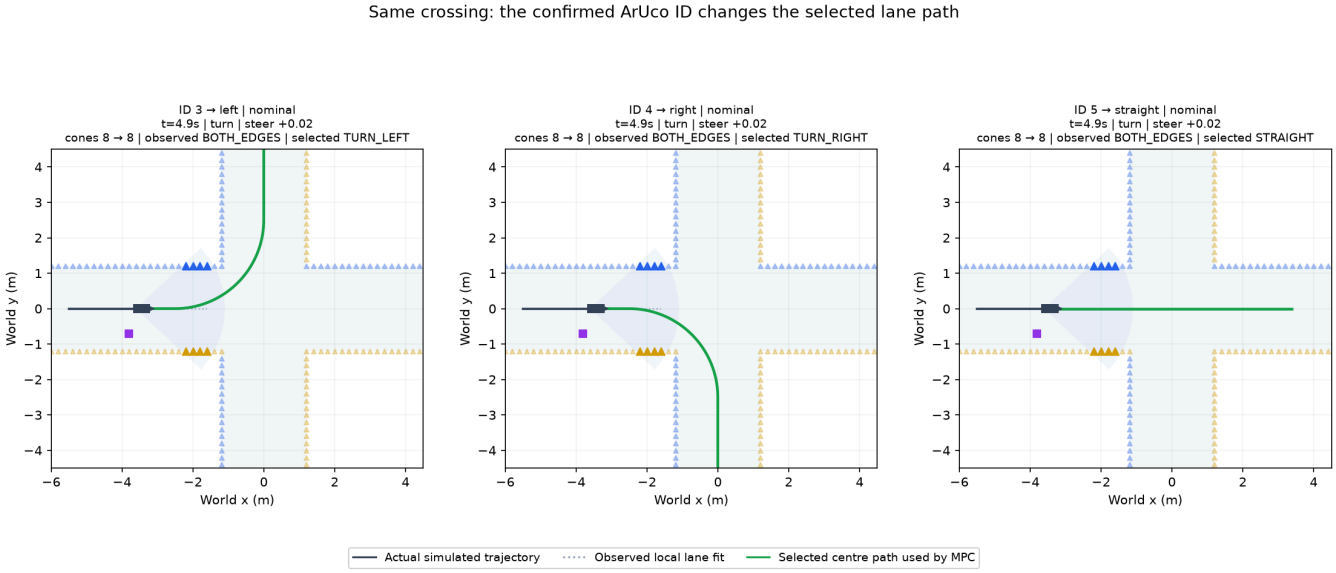
Scenario	Nominal speed	Measured speed	Reduction
Acceleration	0.07158 m	0.07072 m	1.2%
Half achieved speed	0.08314 m	0.08173 m	1.7%
Slow with dropouts	0.07532 m	0.07265 m	3.5%

- **Final-pair lane end (offline):** 20/20 when the pair is visible, but a hidden pair gives false ends → kept offline.
- **Cone-memory U-turn.** The branch experiment completed 10/12 varied simulated layouts. Two triggered clearance stops. The planned 0.50 m radius and 2.2 1/m left curvature limit were simulation assumptions.
- **Cross/T-junction navigation.** With the experimental navigator, 20/22 cross and 22/24 dense open-exit T cases completed in simulation. All six dense blocked-exit cases stopped. The navigator assumes equal-width

perpendicular roads and is disabled by default.

- **Stop latch (sim):** at 0.48 1/m, 6/6 runs on 1.0 m roads stop without contact; 6/6 on 1.5 m complete the left exit. Suite at `488ec30`: 837 passed.

Cone-row association and replay. I separated the incoming cone rows from transverse rows at junctions so the lane fitter could track the approach. The fitter still accepts coherent curves, and obstacle detection keeps all checked cones. Across the same 7,032 synthetic approach frames, accepted misaligned fits fell from 2,265 to 170 and aligned fits rose from 2,528 to 4,628. I also added separate observed-lane and selected-path views in Foxglove and MCAP replay. Evaluation · Implementation

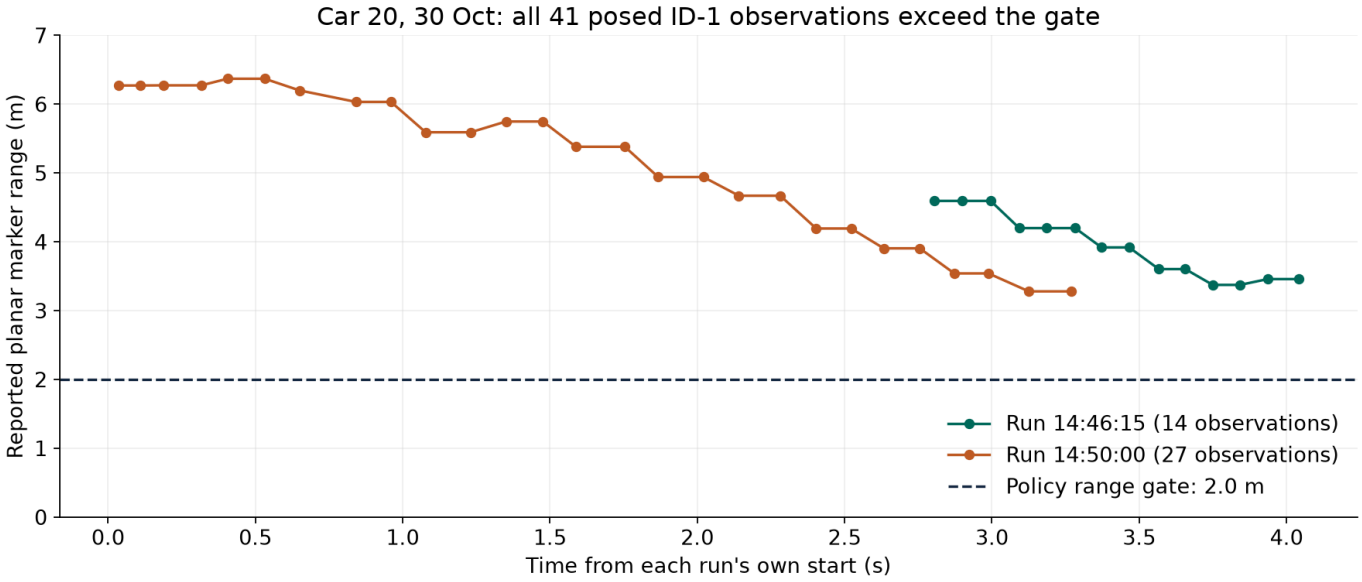


The green lines show the selected MPC paths for three commands at the same simulated crossing. The marker IDs reflect the earlier experimental mapping.

S4. Car 20 deployment and diagnosis

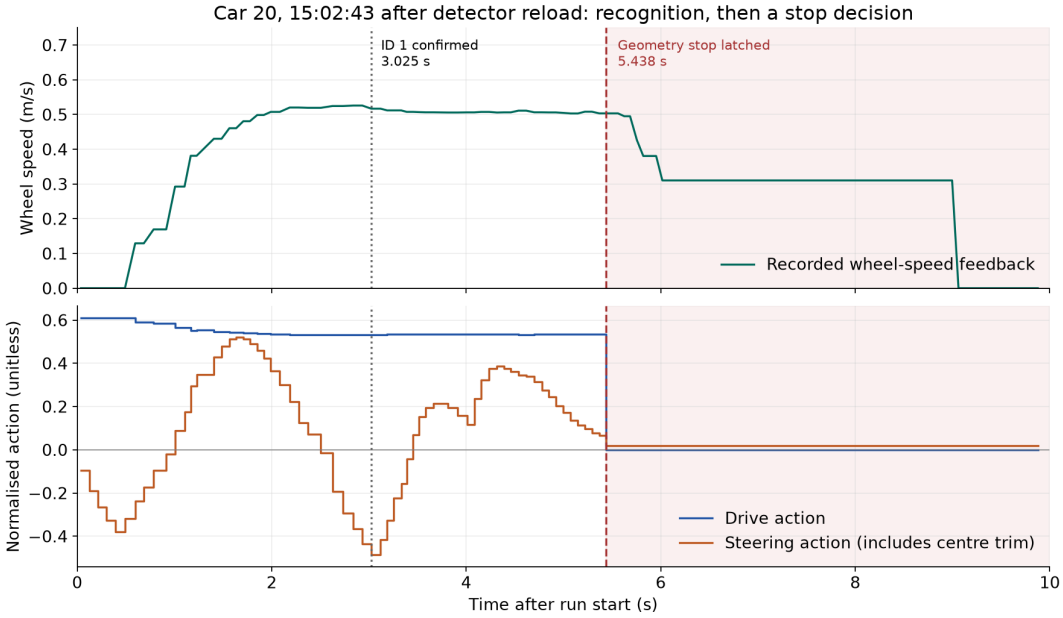
23–26 Oct. Deployment and ArUco recovery. The 23 Oct deployment passed the pinned-interface ROS gate with 186 pytest cases and 189 colcon checks. At startup, all 82 observed drive-and-steering commands were zero in policy state 2. On 26 Oct I traced the missing ArUco stream to disabled camera RGB output. Restoring RGB and CameraInfo yielded 100 advancing ArUco batches in 10 seconds. A later capture confirmed a valid ID 4 pose while the vehicle was disarmed. Deployment record · ArUco recovery

30 Oct. Left-turn diagnosis.



All 41 posed ID-1 observations were 3.3–6.4 m away, outside the 2 m gate. The running detector used a 0.250 m marker size though the print was 0.081 m (range ×3.09); restarting the policy hadn't reloaded it.

- After restarting only the detector: ID 1 confirmed at 3.0 s, then a **geometry stop** at 5.4 s: 0.174 m outside vs 0.182 m needed, heading 4.6° outward. → prepare earlier, tangent admission; limits not relaxed. A zero command isn't a physical stop (wheel-speed feedback was still 0.50 m/s).



The 30 Oct trace marks when the policy confirmed ID 1 and when it latched a geometry stop. Drive command and wheel-speed feedback are shown separately.

Part 4: Car control

4.1 Shared: the car model, controller and limits everyone uses

Everything above car control works in m/s and 1/m; car control is the one block that knows the car (Chris's `CarController`, `policy/control/control.py`, #17).

Interface	Carries	Constraint
(speed, curvature) from the MPC	m/s, 1/m (+ left), brake; 10 Hz	curvature within car 20's +0.48 / −0.92 1/m
Wheel speed	m/s, 10 Hz	unsigned, quantised at low speed, holds its last value 3–4 s after stopping
drive, steer → vehicle	normalised; drive ≤ 0.75	platform slew limits; steer ±0.5 after trim

Car model from system ID (first order plus dead time):

$$v(t) = v_{ss}\left(1 - e^{-(t-t_0-\theta)/\tau}\right), \qquad v_{ss} = K\,(u - u_0)$$

Quantity	Car 8 (29 Sep)	Car 20 (5 Oct)
K (m/s per unit drive)	3.97	9.18 (±10%)
u_0 (drive offset)	0.697	0.475
τ / θ	1.1 s / 0.5 s	2.7 s / 0.2 s

Controller: feedforward for the fast part, low-gain PI for the rest; per-side inverse steering map.

$$u = \mathrm{clip}\left(u_0 + \frac{v_{ref}}{K} + k_p e + I_k,\ 0,\ 0.75\right), \quad I_k = \mathrm{clip}(I_{k-1} + k_i e \Delta t, \pm I_{max}), \quad e = v_{ref} - v$$

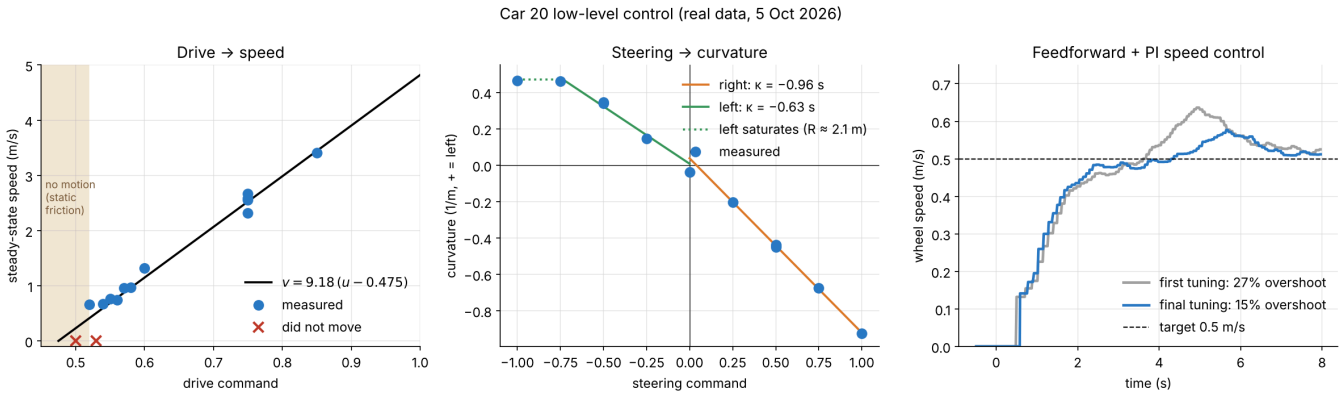
$$s(\kappa) = \begin{cases} (0.038 - \kappa)/0.959 & \kappa < 0 \ \text{(right)} \\ \max\left((0.008 - \kappa)/0.630,\ -0.75\right) & \kappa \geq 0 \ \text{(left, saturates} \approx 0.47\ 1/\mathrm{m}) \end{cases}$$

- Car 20: k_p 0.15, k_i 0.04 (0.2 after the fix), PI off below 0.4 m/s. **No D term:** quantised wheel speed would be amplified. Anti-windup: no integration while clipped; brake bypasses the PI.
- **System consequence:** the left limit (R ≈ 2.1 m) caps the MPC and avoidance: it explains the failed left turns (3.3) and how close two obstacles can be (4.2).

4.2 Chris (Chuhan) Wang — system ID, speed fix and avoidance validation

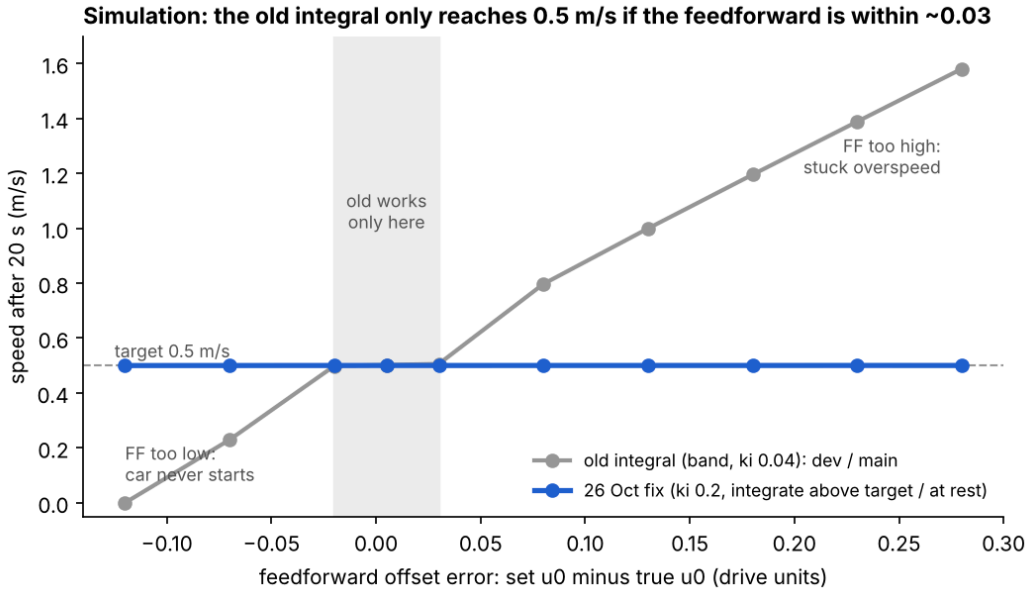
Built: system ID on cars 8 and 20, `CarController` (#17), two-obstacle fixes (#23), the 26 Oct speed fix, avoidance validation on car 20.

S4: system ID (car 20, 5 Oct, with Yifei). One drive level per run, steer 0, 7–8 m straight, bagged; dead-band sweep 0.50–0.60. Battery checked by repeating 0.75 at start and end (1.19 vs 1.14 m/s at 2 s). Steering: PI at 0.5 m/s, κ = yaw rate / speed over the last 3 s; ±0.5 repeated after a reboot within 3%.



(a) drive → speed (shaded: no motion). (b) steer → curvature per side. (c) step to 0.5 m/s: PI v1 27% overshoot, v2 (#17) 15%, ≈ 0.013 m/s steady error.

S3 + S4: the 26 Oct speed fault. The car held 0.87 m/s for a 0.5 target, then (feedforward 0.15) didn't start. Cause: the integral only ran for $|e| < 0.2$ m/s, so a large error froze it.



Sim, true u_0 swept with feedforward fixed at 0.48: old rule reaches 0.5 m/s only for errors –0.02 to +0.03; new rule (integrate when above target or not moving) across –0.12 to +0.28. Trade-off: more overshoot (0.62 vs 0.53 m/s).

Car 20, 26 Oct, fixed	Moving after	Reached 0.5 m/s	Peak	Before stop
15:20:27	0.8 s	1.9 s	0.62 m/s	0.50 m/s
15:21:42	0.4 s	3.2 s	0.57 m/s	0.54 m/s
15:23:01	0.9 s	1.8 s	0.60 m/s	0.52 m/s

Avoidance validation.

- **Sim (15 Oct video, car stopped beside the first of two boards):** two planner bugs (check band widened on both sides; line re-centred on board 1's gap) → all spacings pass with ≥ 0.11 m clearance (#23).
- **Car 20 (19 Oct):** still braked: the lane estimate jumped 0.35 m in 40 ms as a near cone left view (2.1), putting board 1 in the path → block within 0.6 m only if lane and lidar frames agree.
- **How close can two boxes be? (sim, 30 Oct):** left bends worst (≈ 3 m needed at 3 m radius vs 1.2 m straight).
- **Stand test (30 Oct):** RC steered ≈ 2× further than autonomous; ±0.8 gives +60% left travel. The ground re-map (0.91 1/m) wasn't done.

4.3 Yifei Xia — control tests, marker recognition and camera pan

Built: speed and steering tests with Chris (results in 4.1–4.2), ArUco ID test tools (#26), camera pan calibration and a compensation simulation, battery-monitoring study.

S2: heading compensation. With vehicle heading ψ_v and target camera heading ψ_t, wrap w to the shortest angle:

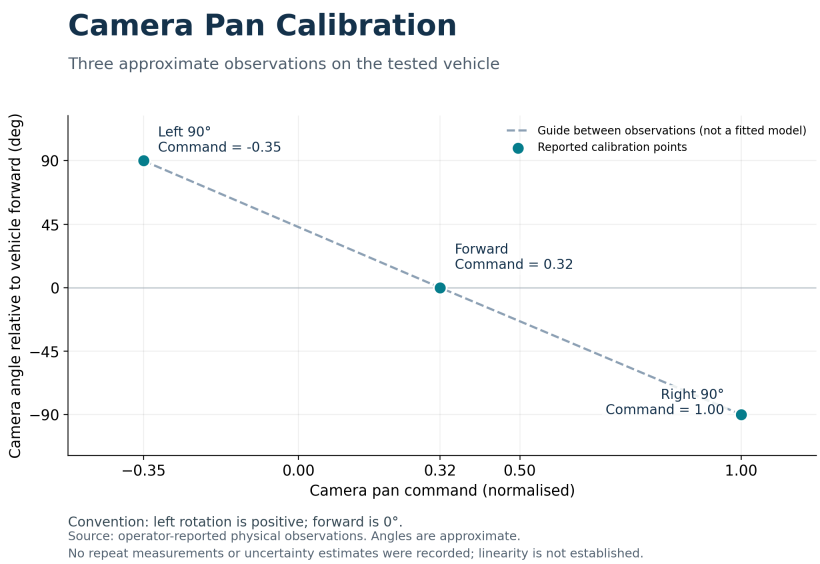
$$\theta_d = w(\psi_t - \psi_v), \qquad u_d = c + g\,\theta_d, \qquad c = 0.32,\ g = -0.43\ \text{per rad}$$

Range ±0.15 about centre (exceeding latches a fault), rate ≤ 0.1 per s, stale IMU (> 0.5 s) → hold.

S3. Interactive simulation (servo τ 0.15 s assumed, delayed heading, 20 s run, CSV export); 11 controller checks pass, no comparative results yet. Offline ArUco decode of stored images: IDs 0, 2, 5, 8 (examples, not accuracy).

S4.

- **Live ArUco (19 Oct):** the detector ran but its image topic had zero publishers. Once the image pipeline ran: `ids=[2]`, `TARGET_SEEN_POSE_OK` (same root cause as 3.3's `--aruco`).
- **Camera pan calibration (30 Oct – 2 Nov):**



Left ≈ +90° at –0.35, forward at 0.32, right ≈ –90° at 1.00: a normalised command is not an angle. No repeats.

- **Battery:** motor 8.4 V NiMH and computer power bank are separate; no voltage source without modification → not implemented; wheel-speed feedback (4.1) absorbs the change instead.
